

TradingAgents Token Usage Optimization

Current state

The pipeline produces accurate, balanced output but uses too many tokens for frequent or backtest.

A test was run to isolate the effect of the debate round setting: the same script was run twice on the same name and date, once at 3 rounds and twice at 1 round, with nothing else changed.

Setting	Input	Output	Total	Calls
3 rounds	247,668	29,299	276,967	26
1 round (run A)	91,308	14,639	105,947	16
1 round (run B)	92,477	15,908	108,385	18

The two 1-round had the same result with difference around 2% and ~106,000 token baseline. Moving from 3 rounds to 1 removes roughly 60-62% of total tokens.

Problem

The cost is dominated by input, not output. The system spends most of its budget re-reading context several times rather than writing it. The input to output ratio is about 8.5 to 1 at 3 rounds and falls to about 5.8 to 1 at 1 round.

Two reasons:

- The four analyst reports are re-sent in full to every debate agent on every round.
- The market analyst re-reads its data and a long instruction prompt across several tool calls.

At 1 round, the market analyst is a large but variable node, measured between about 17,000 and 27,000 input tokens across runs (20-29% of input). The variation comes from its tool loop making a different number of calls run to run. At 3 rounds, the five debate agents dominate instead, together accounting for about 79% of input, because the four-report bundle (about 6,500 tokens) is re-read by all five agents on every round, and the running debate history grows with each round on top of that.

Cost

Token counts translate to API cost as follows, using the models this project runs (gpt-5.5 for the two manager nodes, gpt-5.4-mini for the rest), with a DeepSeek pairing (v4-pro and v4-flash) shown for comparison.

Cost per single run:

Configuration	OpenAI	DeepSeek
3 rounds (old)	\$0.44	\$0.05
1 round (current)	\$0.19	\$0.02

Cost per name for a full backtest (78 weekly runs across 2025 and H1 2026):

Configuration	OpenAI	DeepSeek
3 rounds (old)	\$34.44	\$3.93
1 round (current)	\$14.51	\$1.53

- The round-count cut alone lowers a per-name backtest from about \$34 to about \$15 on OpenAI.
- Moving to the DeepSeek pairing would cut roughly a further ten times on top of that.
- The remaining methods (3, 4, 5) reduce these figures further, most of all for the backtest, where the same run repeats 78 times per name.

Optimization methods

Each method has a score from 0 to 10.

1. Reduce debate rounds from 3 to 1

This is a single value that is changeable through an environment variable with no code edit.

The controlled test showed this alone removes roughly **60-62%** of total tokens. The accuracy cost is expected to be small for this system, because the evidence (the four reports) is fixed before the debate starts, so the extra rounds mostly repeated the same arguments rather than adding new analysis. This will be confirmed or disproved in the backtest by comparing outcomes at 1, 2 and 3 rounds, but 1 round seems like a right default now.

Score: 10/10. Recommendation: use (already applied).

2. Send report digests instead of full reports to the debate agents

The four full analyst reports (about 6,500 tokens together) were re-sent to all five debate agents on every turn. The change replaces them with short digests.

It is implemented and measured: the five debate agents dropped from about 45,500 to about 17,800 input tokens, a **61%** cut, taking the whole run from about 106,000 to about 86,000 tokens. The managers and trader do not read the reports, so the change stays contained to five prompts. The dollar saving is small, about \$0.02 per run because the debators run on the cheaper model, and it grows at higher round counts.

Score: 8/10. Recommendation: use (already applied).

3. Convert the market analyst to pre-fetched data and trim its prompt

The market analyst re-reads a long indicator instruction prompt and its accumulating data across multiple model calls, which is also why its cost varies between runs.

Pre-computing the data and injecting it once, then shortening the indicator instructions, would replace these multiple calls with just one, remove the re-reads and make the cost consistent.

This pattern is already proven in the codebase: the sentiment analyst was redesigned the same way and now runs in a single cheap call.

Score: 8/10. Recommendation: use (not done yet).

4. Pre-download and trim raw data before the run

Raw data (price history, financial statements, news) is fetched live and injected at full size. The fundamentals analyst alone ingests about 6,000 tokens of statements and price history adds about 2,300.

Downloading and trimming this once before the run, and lowering the news article limit, gives not very significant savings per run. However, the value is large for the backtest, where the same data would otherwise be re-fetched every week across about 78 weeks.

Score: 7/10. Recommendation: use (not done yet)..

5. Remove duplicate passing of the last argument in the debate agents

Each debate agent receives the most recent argument twice, once inside the running history and once as a separate field. Removing the duplicate is free and safe but saves little.

Score: 4/10. Recommendation: optional.

Summary

Method	Score	Recommendation
1. Reduce debate rounds from 3 to 1	10/10	Use (already applied)
2. Report digests for the debate agents	8/10	Use (already applied)
3. Market analyst: pre-fetch data and trim prompt	8/10	Use
4. Pre-download and trim raw data	7/10	Use
5. Remove duplicate argument passing	4/10	Optional

- Methods 1 and 2 are implemented and measured. Together they take a run from about 276,967 tokens (3 rounds, full reports) to about 86,000, a reduction of roughly **70%**.
- Methods 3 and 4 are not yet built, with method 4 being the most significant for backtesting.
- Method 5 is not too important.